Rockchip RGB and MCU Interface Developer Guide

ID: RK-YH-YF-483

Release Version: V1.8.0

Release Date: 2026-01-04

Security Level: ☐Top-Secret ☐Secret ☐Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

This document primarily introduces the debugging and verification guide for low-speed display interfaces on the Rockchip platform.

Intended Audience

This document (this guide) is mainly applicable to the following engineers:

Technical Support Engineers

Software Development Engineers

Hardware Development Engineers

Revision History

Version	Author	Date	Description
V1.0.0	Ding Lingsong	2023-07-01	Initial release
V1.1.0	Ding Lingsong	2023-07-15	Added detailed descriptions for mcu-timing and display-timings
V1.2.0	Ding Lingsong	2023-09-05	Revised configuration instructions for mcu-timing and display-timings
V1.3.0	Ding Lingsong	2024-03-21	Added support for RK3576
V1.4.0	Ding Lingsong	2024-06-26	Added description for SPI RGB Panel
V1.5.0	Ding Lingsong	2024-08-27	Added support for RK3506 and related instructions for mcu read functionality
V1.6.0	Chen Chaoyi	2025-04-24	Added support for RV1126B
V1.7.0	Chen Chaoyi	2025-06-18	Added hardware connection instructions for RV1126B-P
V1.7.1	Huang Tao	2025-07-16	Revised formatting
V1.8.0	Ding Lingsong	2026-01-04	Added descriptions for RGB Delayline configuration and MCU PSR Mode features

Table of Contents

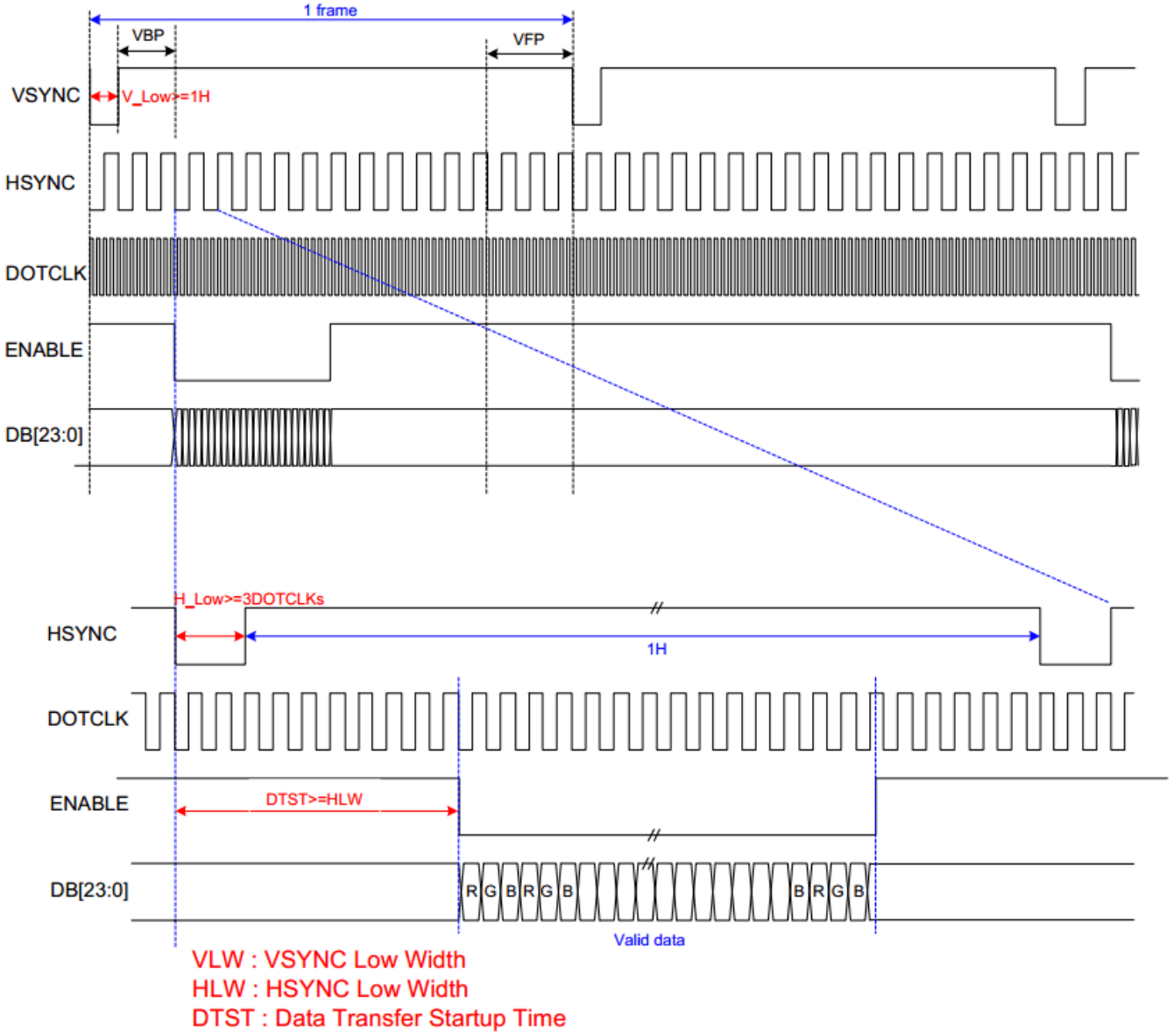
Rockchip RGB and MCU Interface Developer Guide

1. Concepts
   - 1.1 RGB Interface
     - 1.1.1 DE Mode
     - 1.1.2 SYNC Mode
   - 1.2 MCU Interface
     - 1.2.1 Write Timing
     - 1.2.2 Read Timing
     - 1.2.3 Bypass and Normal Modes
2. RK Platform Support Status
3. Hardware Connection
   - 3.1 RK3562/RK3576/RK3506/RV1126B Platforms
   - 3.2 RK3568 Platform
   - 3.3 RK312X/PX3SE/RK3288/RK3308B/RK3308BS/RK3328/RK3326/PX30/RV1109/RV1126 Platforms
   - 3.4 RK1808/RV1106 Platform
   - 3.5 RV1103 Platform
   - 3.6 RV1126B-P Platform
4. Software Configuration
   - 4.1 Display Path
   - 4.2 Panel Configuration
     - 4.2.1 SPI Initialization Configuration
   - 4.3 RGB Interface
     - 4.3.1 RGB Delayline Configuration
   - 4.4 MCU Interface
     - 4.4.1 MCU Bypass Timing Configuration
     - 4.4.2 MCU Frame Write/Read
     - 4.4.3 MCU PSR Mode
5. Debugging Procedure
6. FAQ
   - 6.1 The RGB display can show images, but there is noise on the screen or display misalignment occurs

1. Concepts

1.1 RGB Interface

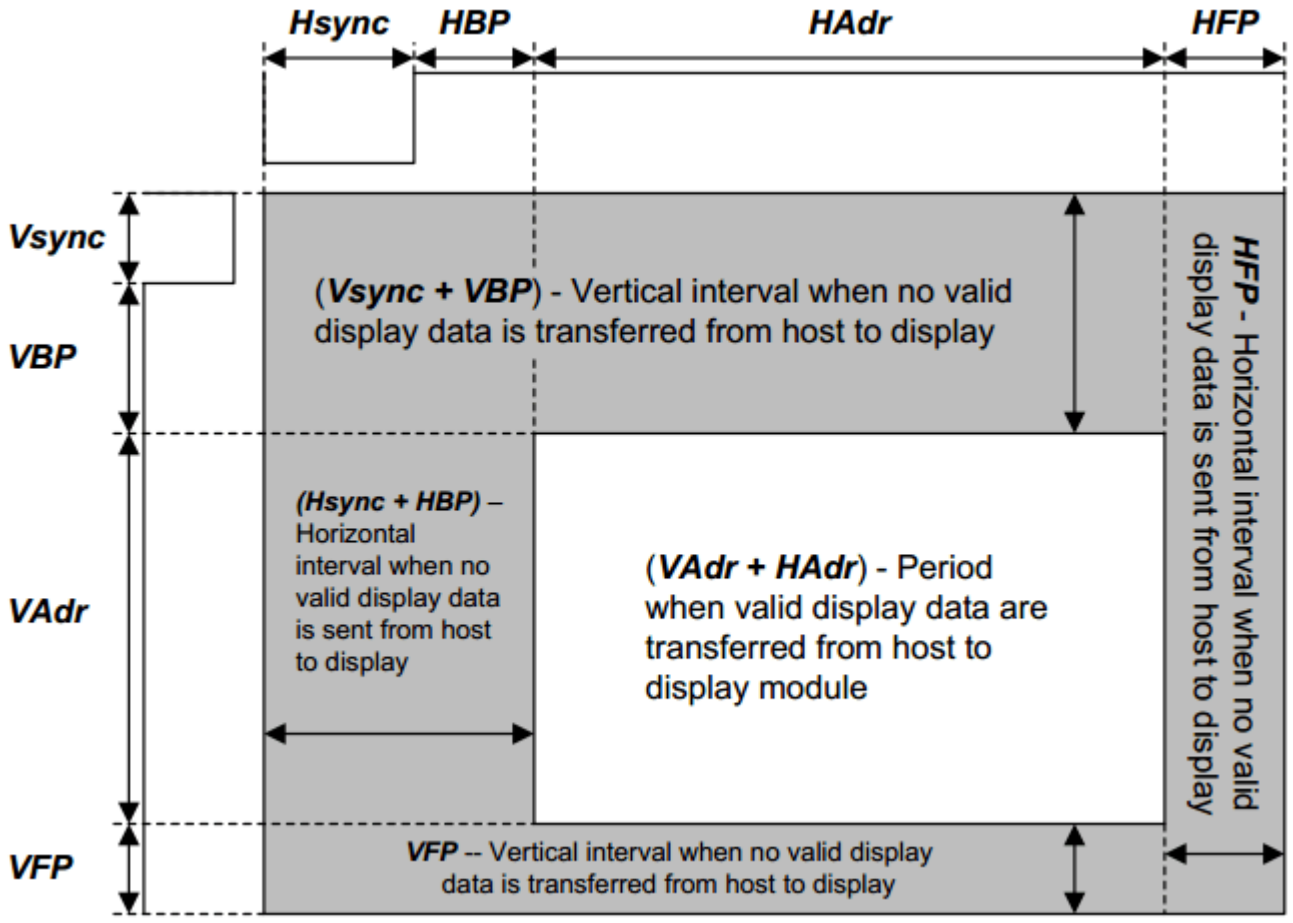
The RGB interface, also known as the DPI (Display Pixel Interface), utilizes four pins for synchronization signals: Vsync, Hsync, Den (Enable), and DCLK (Dotclk). Based on the synchronization method, it can be categorized into DE mode and SYNC mode. The Rockchip platform RGB interface output timing is compatible with both modes simultaneously.



1.1.1 DE Mode

The validity of DB[23:0] data is determined solely by the Den signal; data is valid when the signal is low, and invalid otherwise.

1.1.2 SYNC Mode

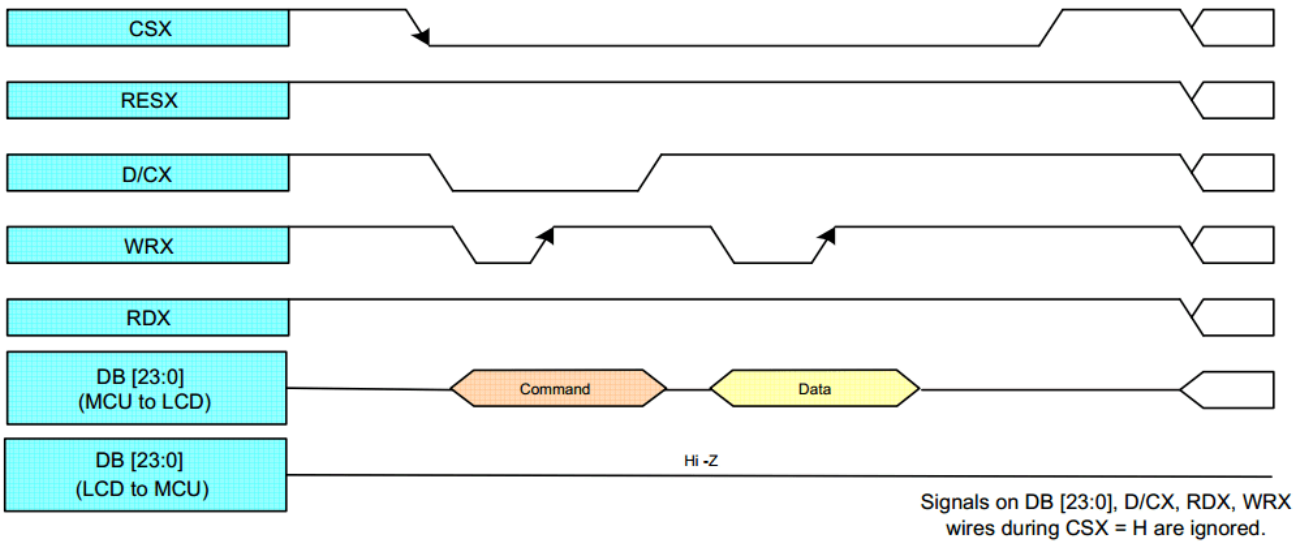


DB[23:0] data is synchronized by Vsync and Hsync signals, scanning data according to the timing diagram above.

1.2 MCU Interface

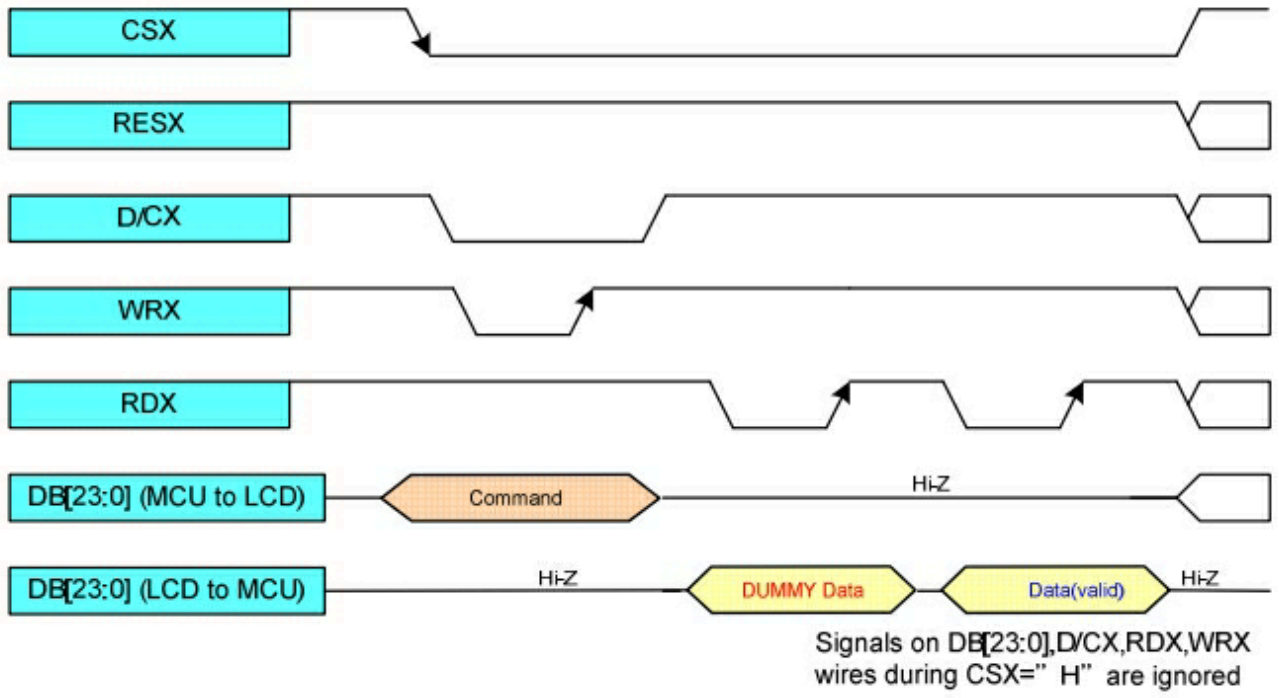
The MCU interface, also known as the DBI interface or 8080 interface, supports bidirectional communication between TX and RX ends. It utilizes four synchronization signals: RS (CSX), CSN (D/CX), WEN (WRX), and REN (RDX). The RK platform supports only the TX function of the MCU interface.

1.2.1 Write Timing



The CSX, D/CX, and WRX pins are pulled low sequentially. During the period when DB[23:0] data is valid, the WRX signal is first pulled low and then pulled high.

1.2.2 Read Timing



The CSX, D/CX, and RDX pins are pulled low sequentially. During the validity period of the DB[23:0] data, the RDX signal is pulled low first and then pulled high.

- Before a read operation, a write operation must be performed to transmit the target Register address to the Panel.
- The data returned from the first read operation is invalid; valid data is obtained starting from the second read operation.
- MCU read operations are typically used for debugging during panel development, implementing multi-

display compatibility by distinguishing panel IDs, and for the frame read functionality described later in this document.

1.2.3 Bypass and Normal Modes

- Bypass Mode: The interface operates in bypass mode when instructions are transmitted between the MCU and the panel via write/read operations.

Transmitting init/deinit instructions according to the panel-init-sequence/panel-exit-sequence via the MCU interface, as well as performing frame write/read operations (see relevant descriptions in the Software Configuration section below), are all classified as bypass mode.

- Normal Mode: This mode involves the main controller transmitting image data to the panel RAM via the MCU interface for normal display. The system typically enters this mode after the panel-init-sequence transmission is completed and successful panel initialization is confirmed.

Write/read operations can occur in bypass mode, whereas normal mode supports only write operations.

2. RK Platform Support Status

SoC Platform	RGB	RGB Delayline	MCU	MCU Read	MCU PSR	VOP Version	Video Port Path (for VOP 2.0)	Output Mode
RK1808	Y	N	Y	N	Y	VOP 1.0		RGB666/RGB565
RK312X/PX3SE	Y	N	N	N	N	VOP 1.0		RGB888/RGB666/RGB565
RK3288	Y	N	Y	N	Y	VOP 1.0		RGB888/RGB666/RGB565/RGB3x8
RK3308B/RK3308BS	Y	N	Y	N	Y	VOP 1.0		RGB888/RGB666/RGB565/RGB3x8
RK3326/PX30	Y	N	Y	N	Y	VOP 1.0		RGB888/RGB666/RGB565
RK3506	Y	Y	Y	Y	Y	VOP 1.0		RGB888/RGB666/RGB565/RGB3x8/RGB3x6/RGB2x8
RK3562	Y	N	Y	N	Y	VOP 2.0	VP0	RGB888/RGB666/RGB565/RGB3x8
RK3568	Y	N	N	N	N	VOP 2.0	VP2	RGB888/RGB666/RGB565
RK3576	Y	Y	Y	N	Y	VOP 2.0	VP1/VP2	RGB888/RGB666/RGB565/RGB3x8/RGB3x6/RGB2x8
RV1103	Y	N	Y	N	Y	VOP 1.0		RGB3x8
RV1106	Y	N	Y	N	Y	VOP 1.0		RGB666/RGB565/RGB3x8
RV1109/RV1126	Y	N	Y	N	Y	VOP 1.0		RGB888/RGB666/RGB565/RGB3x8
RV1126B	Y	Y	Y	Y	Y	VOP 1.0		RGB888/RGB666/RGB565/RGB3x8/RGB3x6/RGB2x8

Note: For concepts related to VOP and VP (Video Port) mentioned above, refer to the document "Rockchip_Developer_Guide_DRM_Display_Driver_EN".

3. Hardware Connection

3.1 RK3562/RK3576/RK3506/RV1126B Platforms

Component Name	Pin Name	RGB888 (MCU)	RGB666 (MCU)	RGB565 (MCU)	RGB3x8 (MCU)	RGB3x6 (MCU)	RGB2x8 (MCU)
DCLK	VO_LCDC_CLK	DCLK(RS)	DCLK(RS)	DCLK(RS)	DCLK(RS)	DCLK(RS)	DCLK(RS)
VSYNC	VO_LCDC_VSYNC	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)
HSYNC	VO_LCDC_HSYNC	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)
DEN	VO_LCDC_DEN	DEN(RDN)	DEN(RDN)	DEN(RDN)	DEN(RDN)	DEN(RDN)	DEN(RDN)
R7_D23	VO_LCDC_D23	√	√	√	√ (D7_m1)	√ (D5_m1)	√ (D7_m1)
R6_D22	VO_LCDC_D22	√	√	√	√ (D6_m1)	√ (D4_m1)	√ (D6_m1)
R5_D21	VO_LCDC_D21	√	√	√	√ (D5_m1)	√ (D3_m1)	√ (D5_m1)
R4_D20	VO_LCDC_D20	√	√	√	√ (D4_m1)	√ (D2_m1)	√ (D4_m1)
R3_D19	VO_LCDC_D19	√	√	√	√ (D3_m1)	√ (D1_m1)	√ (D3_m1)
R2_D18	VO_LCDC_D18	√	√	×	×	×	×
R1_D17	VO_LCDC_D17	√	×	×	×	×	×
R0_D16	VO_LCDC_D16	√	×	×	×	×	×
G7_D15	VO_LCDC_D15	√	√	√	√ (D2_m1)	√ (D0_m1)	√ (D2_m1)
G6_D14	VO_LCDC_D14	√	√	√	√ (D1_m1)	×	√ (D1_m1)
G5_D13	VO_LCDC_D13	√	√	√	√ (D0_m1)	×	√ (D0_m1)
G4_D12	VO_LCDC_D12	√	√	√	√ (D7_m0)	√ (D5_m0)	√ (D7_m0)
G3_D11	VO_LCDC_D11	√	√	√	√ (D6_m0)	√ (D4_m0)	√ (D6_m0)
G2_D10	VO_LCDC_D10	√	√	√	√ (D5_m0)	√ (D3_m0)	√ (D5_m0)
G1_D9	VO_LCDC_D9	√	×	×	×	×	×
G0_D8	VO_LCDC_D8	√	×	×	×	×	×
B7_D7	VO_LCDC_D7	√	√	√	√ (D4_m0)	√ (D2_m0)	√ (D4_m0)
B6_D6	VO_LCDC_D6	√	√	√	√ (D3_m0)	√ (D1_m0)	√ (D3_m0)
B5_D5	VO_LCDC_D5	√	√	√	√ (D2_m0)	√ (D0_m0)	√ (D2_m0)
B4_D4	VO_LCDC_D4	√	√	√	√ (D1_m0)	×	√ (D1_m0)
B3_D3	VO_LCDC_D3	√	√	√	√ (D0_m0)	×	√ (D0_m0)
B2_D2	VO_LCDC_D2	√	√	×	×	×	×
B1_D1	VO_LCDC_D1	√	×	×	×	×	×
B0_D0	VO_LCDC_D0	√	×	×	×	×	×

3.2 RK3568 Platform

Component Name	Pin Name	RGB888	RGB666	RGB565
DCLK	LCDC_CLK	DCLK	DCLK	DCLK
VSYNC	LCDC_VSYNC	VSYNC	VSYNC	VSYNC
HSYNC	LCDC_HSYNC	HSYNC	HSYNC	HSYNC
DEN	LCDC_DEN	DEN	DEN	DEN
R7_D23	LCDC_D23	√	√	√
R6_D22	LCDC_D22	√	√	√
R5_D21	LCDC_D21	√	√	√
R4_D20	LCDC_D20	√	√	√
R3_D19	LCDC_D19	√	√	√
R2_D18	LCDC_D18	√	√	×
R1_D17	LCDC_D17	√	×	×
R0_D16	LCDC_D16	√	×	×
G7_D15	LCDC_D15	√	√	√
G6_D14	LCDC_D14	√	√	√
G5_D13	LCDC_D13	√	√	√
G4_D12	LCDC_D12	√	√	√
G3_D11	LCDC_D11	√	√	√
G2_D10	LCDC_D10	√	√	√
G1_D9	LCDC_D9	√	×	×
G0_D8	LCDC_D8	√	×	×
B7_D7	LCDC_D7	√	√	√
B6_D6	LCDC_D6	√	√	√
B5_D5	LCDC_D5	√	√	√
B4_D4	LCDC_D4	√	√	√
B3_D3	LCDC_D3	√	√	√
B2_D2	LCDC_D2	√	√	×

Component Name	Pin Name	RGB888	RGB666	RGB565
B1_D1	LCDC_D1	√	×	×
B0_D0	LCDC_D0	√	×	×

3.3 RK312X/PX3SE/RK3288/RK3308B/RK3308BS/RK3328/RK3326/PX30/RV1109/RV1126 Platforms

Component Name	Pin Name	RGB888 (MCU)	RGB666 (MCU)	RGB666_CPADHI (MCU)	RGB565 (MCU)	RGB565_CPADHI (MCU)	RGB3x8 (MCU)
DCLK	LCDC_CLK/LCD_CLK	DCLK(RS)	DCLK(RS)	DCLK(RS)	DCLK(RS)	DCLK(RS)	DCLK(RS)
VSYNC	LCDC_VSYNC/LCD_VSYNC	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)
HSYNC	LCDC_HSYNC/LCD_HSYNC	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)
DEN	LCDC_DEN/LCD_DEN	DEN(RDN)	DEN(RDN)	DEN(RDN)	DEN(RDN)	DEN(RDN)	DEN(RDN)
R7_D23	LCDC_D23/LCD_D23	√	×	√	×	√	×
R6_D22	LCDC_D22/LCD_D22	√	×	√	×	√	×
R5_D21	LCDC_D21/LCD_D21	√	×	√	×	√	×
R4_D20	LCDC_D20/LCD_D20	√	×	√	×	√	×
R3_D19	LCDC_D19/LCD_D19	√	×	√	×	√	×
R2_D18	LCDC_D18/LCD_D18	√	×	√	×	×	×
R1_D17	LCDC_D17/LCD_D17	√	√	×	×	×	×
R0_D16	LCDC_D16/LCD_D16	√	√	×	×	×	×
G7_D15	LCDC_D15/LCD_D15	√	√	√	√	√	×
G6_D14	LCDC_D14/LCD_D14	√	√	√	√	√	×
G5_D13	LCDC_D13/LCD_D13	√	√	√	√	√	×
G4_D12	LCDC_D12/LCD_D12	√	√	√	√	√	×
G3_D11	LCDC_D11/LCD_D11	√	√	√	√	√	×
G2_D10	LCDC_D10/LCD_D10	√	√	√	√	√	×
G1_D9	LCDC_D9/LCD_D9	√	√	×	√	×	×
G0_D8	LCDC_D8/LCD_D8	√	√	×	√	×	×
B7_D7	LCDC_D7/LCD_D7	√	√	√	√	√	√
B6_D6	LCDC_D6/LCD_D6	√	√	√	√	√	√
B5_D5	LCDC_D5/LCD_D5	√	√	√	√	√	√
B4_D4	LCDC_D4/LCD_D4	√	√	√	√	√	√
B3_D3	LCDC_D3/LCD_D3	√	√	√	√	√	√
B2_D2	LCDC_D2/LCD_D2	√	√	√	√	×	√
B1_D1	LCDC_D1/LCD_D1	√	√	×	√	×	√
B0_D0	LCDC_D0/LCD_D0	√	√	×	√	×	√

3.4 RK1808/RV1106 Platform

Component Name	Pin Name	RGB666 (MCU)	RGB565 (MCU)	RGB565 (MCU)	RGB3x8 (MCU)
DCLK	LCDC_CLK/LCD_CLK	DCLK(RS)	DCLK(RS)	DCLK(RS)	DCLK(RS)
VSYNC	LCDC_VSYNC/LCD_VSYNC	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)
HSYNC	LCDC_HSYNC/LCD_HSYNC	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)
DEN	LCDC_DEN/LCD_DEN	DEN(RDN)	DEN(RDN)	DEN(RDN)	DEN(RDN)
R5_D17	LCDC_D17/LCD_D17	√	×	×	×
R4_D16	LCDC_D16/LCD_D16	√	×	×	×
R3_D15	LCDC_D15/LCD_D15	√	√	√	×
R2_D14	LCDC_D14/LCD_D14	√	√	√	×
R1_D13	LCDC_D13/LCD_D13	√	√	√	×
R0_D12	LCDC_D12/LCD_D12	√	√	√	×
G5_D11	LCDC_D11/LCD_D11	√	√	√	×
G4_D10	LCDC_D10/LCD_D10	√	√	√	×
G3_D9	LCDC_D9/LCD_D9	√	√	√	×
G2_D8	LCDC_D8/LCD_D8	√	√	√	×
G1_D7	LCDC_D7/LCD_D7	√	√	√	√
G0_D6	LCDC_D6/LCD_D6	√	√	√	√
B5_D5	LCDC_D5/LCD_D5	√	√	√	√
B4_D4	LCDC_D4/LCD_D4	√	√	√	√
B3_D3	LCDC_D3/LCD_D3	√	√	√	√
B2_D2	LCDC_D2/LCD_D2	√	√	√	√
B1_D1	LCDC_D1/LCD_D1	√	√	√	√
B0_D0	LCDC_D0/LCD_D0	√	√	√	√

3.5 RV1103 Platform

Component Name	Pin Name	RGB3x8 (MCU)
DCLK	LCDC_CLK	DCLK(RS)
VSYNC	LCDC_VSYNC	VSYNC(CSN)
HSYNC	LCDC_HSYNC	HSYNC(WRN)
DEN	LCDC_DEN	DEN(RDN)
D7	LCDC_D7	√
D6	LCDC_D6	√
D5	LCDC_D5	√
D4	LCDC_D4	√
D3	LCDC_D3	√
D2	LCDC_D2	√
D1	LCDC_D1	√
D0	LCDC_D0	√

3.6 RV1126B-P Platform

For the RV1126B-P platform, the concept of a data map is introduced to maintain consistency with the hardware connection scheme of the earlier RV1126 platform. By configuring different data maps, data can be output on different hardware pins.

The complete definition of the data map can be found in `include/dt-bindings/display/rockchip_vop.h`, as follows:

```
/* mcu_data[23:0] */
#define ROCKCHIP_MCU_DATA_MAP_DATA_1x24        0
/*
 * { mcu_data[7:3], 3'b0, mcu_data[2:0], mcu_data[7:5], 2'b0, mcu_data[4:0], 3'b0 }
 * That is:
 *    LCDC_D23 -> MCU_D7_M1
 *    LCDC_D22 -> MCU_D6_M1
 *    LCDC_D21 -> MCU_D5_M1
 *    LCDC_D20 -> MCU_D4_M1
 *    LCDC_D19 -> MCU_D3_M1
 *    LCDC_D18 -> N/A
 *    LCDC_D17 -> N/A
 *    LCDC_D16 -> N/A
 *    LCDC_D15 -> MCU_D2_M1
 *    LCDC_D14 -> MCU_D1_M1
 *    LCDC_D13 -> MCU_D0_M1
```

```
 *    LCDC_D12 -> MCU_D7_M0
 *    LCDC_D11 -> MCU_D6_M0
 *    LCDC_D10 -> MCU_D5_M0
 *    LCDC_D9  -> N/A
 *    LCDC_D8  -> N/A
 *    LCDC_D7  -> MCU_D4_M0
 *    LCDC_D6  -> MCU_D3_M0
 *    LCDC_D5  -> MCU_D2_M0
 *    LCDC_D4  -> MCU_D1_M0
 *    LCDC_D3  -> MCU_D0_M0
 *    LCDC_D2  -> N/A
 *    LCDC_D1  -> N/A
 *    LCDC_D0  -> N/A
 */
#define ROCKCHIP_MCU_DATA_MAP_DATA_WITH_DUMMY_0    1
```

For the standard RV1126B, the default MCU hardware connection method is as follows, corresponding to `ROCKCHIP_MCU_DATA_MAP_DATA_WITH_DUMMY_0`.

Component Name	Pin Name	RGB888 (MCU)	RGB666 (MCU)	RGB565 (MCU)	RGB3x8 (MCU)	RGB3x6 (MCU)	RGB2x8 (MCU)
DCLK	VO_LCDC_CLK	DCLK(RS)	DCLK(RS)	DCLK(RS)	DCLK(RS)	DCLK(RS)	DCLK(RS)
VSYNC	VO_LCDC_VSYNC	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)
HSYNC	VO_LCDC_HSYNC	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)
DEN	VO_LCDC_DEN	DEN(RDN)	DEN(RDN)	DEN(RDN)	DEN(RDN)	DEN(RDN)	DEN(RDN)
R7_D23	VO_LCDC_D23	√	√	√	√ (D7_m1)	√ (D5_m1)	√ (D7_m1)
R6_D22	VO_LCDC_D22	√	√	√	√ (D6_m1)	√ (D4_m1)	√ (D6_m1)
R5_D21	VO_LCDC_D21	√	√	√	√ (D5_m1)	√ (D3_m1)	√ (D5_m1)
R4_D20	VO_LCDC_D20	√	√	√	√ (D4_m1)	√ (D2_m1)	√ (D4_m1)
R3_D19	VO_LCDC_D19	√	√	√	√ (D3_m1)	√ (D1_m1)	√ (D3_m1)
R2_D18	VO_LCDC_D18	√	√	×	×	×	×
R1_D17	VO_LCDC_D17	√	×	×	×	×	×
R0_D16	VO_LCDC_D16	√	×	×	×	×	×
G7_D15	VO_LCDC_D15	√	√	√	√ (D2_m1)	√ (D0_m1)	√ (D2_m1)
G6_D14	VO_LCDC_D14	√	√	√	√ (D1_m1)	×	√ (D1_m1)
G5_D13	VO_LCDC_D13	√	√	√	√ (D0_m1)	×	√ (D0_m1)
G4_D12	VO_LCDC_D12	√	√	√	√ (D7_m0)	√ (D5_m0)	√ (D7_m0)
G3_D11	VO_LCDC_D11	√	√	√	√ (D6_m0)	√ (D4_m0)	√ (D6_m0)
G2_D10	VO_LCDC_D10	√	√	√	√ (D5_m0)	√ (D3_m0)	√ (D5_m0)
G1_D9	VO_LCDC_D9	√	×	×	×	×	×
G0_D8	VO_LCDC_D8	√	×	×	×	×	×
B7_D7	VO_LCDC_D7	√	√	√	√ (D4_m0)	√ (D2_m0)	√ (D4_m0)
B6_D6	VO_LCDC_D6	√	√	√	√ (D3_m0)	√ (D1_m0)	√ (D3_m0)
B5_D5	VO_LCDC_D5	√	√	√	√ (D2_m0)	√ (D0_m0)	√ (D2_m0)
B4_D4	VO_LCDC_D4	√	√	√	√ (D1_m0)	×	√ (D1_m0)
B3_D3	VO_LCDC_D3	√	√	√	√ (D0_m0)	×	√ (D0_m0)
B2_D2	VO_LCDC_D2	√	√	×	×	×	×
B1_D1	VO_LCDC_D1	√	×	×	×	×	×
B0_D0	VO_LCDC_D0	√	×	×	×	×	×

For the RV1126B-P, to ensure MCU hardware output remains consistent with the earlier RV1126, set the connection mode to `ROCKCHIP_MCU_DATA_MAP_DATA_1x24`, as shown below:

Component Name	Pin Name	RGB888 (MCU)	RGB666 (MCU)	RGB666_CPADHI (MCU)	RGB565 (MCU)	RGB565_CPADHI (MCU)	RGB3x8 (MCU)
DCLK	LCDC_CLK/LCD_CLK	DCLK(RS)	DCLK(RS)	DCLK(RS)	DCLK(RS)	DCLK(RS)	DCLK(RS)
VSYNC	LCDC_VSYNC/LCD_VSYNC	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)	VSYNC(CSN)
HSYNC	LCDC_HSYNC/LCD_HSYNC	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)	HSYNC(WRN)
DEN	LCDC_DEN/LCD_DEN	DEN(RDN)	DEN(RDN)	DEN(RDN)	DEN(RDN)	DEN(RDN)	DEN(RDN)
R7_D23	LCDC_D23/LCD_D23	√	×	√	×	√	×
R6_D22	LCDC_D22/LCD_D22	√	×	√	×	√	×
R5_D21	LCDC_D21/LCD_D21	√	×	√	×	√	×
R4_D20	LCDC_D20/LCD_D20	√	×	√	×	√	×
R3_D19	LCDC_D19/LCD_D19	√	×	√	×	√	×
R2_D18	LCDC_D18/LCD_D18	√	×	√	×	×	×
R1_D17	LCDC_D17/LCD_D17	√	√	×	×	×	×
R0_D16	LCDC_D16/LCD_D16	√	√	×	×	×	×
G7_D15	LCDC_D15/LCD_D15	√	√	√	√	√	×
G6_D14	LCDC_D14/LCD_D14	√	√	√	√	√	×
G5_D13	LCDC_D13/LCD_D13	√	√	√	√	√	×
G4_D12	LCDC_D12/LCD_D12	√	√	√	√	√	×
G3_D11	LCDC_D11/LCD_D11	√	√	√	√	√	×
G2_D10	LCDC_D10/LCD_D10	√	√	√	√	√	×
G1_D9	LCDC_D9/LCD_D9	√	√	×	√	×	×
G0_D8	LCDC_D8/LCD_D8	√	√	×	√	×	×
B7_D7	LCDC_D7/LCD_D7	√	√	√	√	√	√
B6_D6	LCDC_D6/LCD_D6	√	√	√	√	√	√
B5_D5	LCDC_D5/LCD_D5	√	√	√	√	√	√
B4_D4	LCDC_D4/LCD_D4	√	√	√	√	√	√
B3_D3	LCDC_D3/LCD_D3	√	√	√	√	√	√
B2_D2	LCDC_D2/LCD_D2	√	√	√	√	×	√
B1_D1	LCDC_D1/LCD_D1	√	√	×	√	×	√
B0_D0	LCDC_D0/LCD_D0	√	√	×	√	×	√

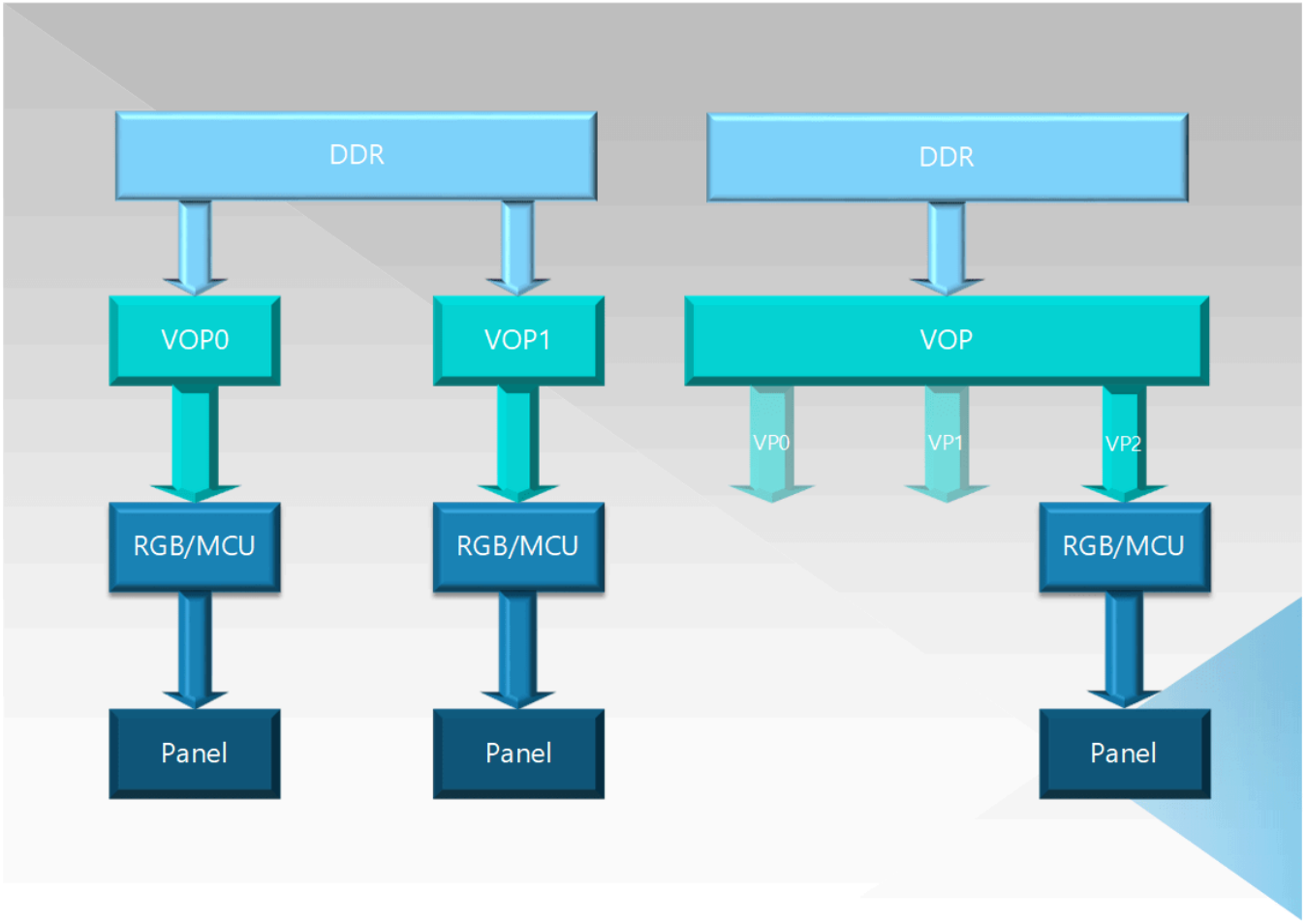
Generally, no additional configuration is required for RV1126B and RV1126B-P. However, if modification is necessary, it can be configured in the DTS.

The following example shows the configuration for RV1126B using the earlier RV1126 hardware connection method (for RV1126B-P, this configuration is set by default):

```
&rgb {
    /*
     * RV1126 compatible pin output mode 0.
     * MCU: ROCKCHIP_MCU_DATA_MAP_DATA_1x24
     */
    rockchip,data-map-mode = <0>;
};
```

4. Software Configuration

4.1 Display Path



VOP (Video Output Processor) is the display processing unit on the Rockchip platform. There are two architectures, VOP 1.0 and VOP 2.0, which primarily differ in their support methods for multiple displays. For detailed information, refer to the document "Rockchip_Developer_Guide_DRM_Display_Driver_CN". The block diagrams on the left and right correspond to the display paths for the RGB/MCU interfaces under the VOP 1.0 and VOP 2.0 architectures, respectively. The VOP reads image data from DDR for processing and then sends it to the RGB/MCU display interface. The interface module converts the image data into a protocol-compliant data stream, which is finally transmitted to the screen for display.

4.2 Panel Configuration

The RGB panel Driver can refer to the implementation in `drivers/gpu/drm/panel/panel-simple.c`. The following shows a typical panel node configuration:

```
/ {
    panel: panel {
        compatible = "simple-panel";
        bus-format = <MEDIA_BUS_FMT_RGB888_1X24>;
        backlight = <&backlight>;
```

```
        enable-gpios = <&gpio3 RK_PA6 GPIO_ACTIVE_LOW>;
        enable-delay-ms = <20>;
        reset-gpios = <&gpio3 RK_PB0 GPIO_ACTIVE_LOW>;
        reset-delay-ms = <10>;
        status = "okay";

        display-timings {
          native-mode = <&fx070_dhm11boe_timing>;

          fx070_dhm11boe_timing: timing0 {
            clock-frequency = <50000000>;
            hactive = <1024>;
            vactive = <600>;
            hback-porch = <140>;
            hfront-porch = <160>;
            vback-porch = <20>;
            vfront-porch = <20>;
            hsync-len = <20>;
            vsync-len = <2>;
            hsync-active = <0>;
            vsync-active = <0>;
            de-active = <0>;
            pixelclk-active = <0>;
          };
        };

        port {
          panel_in_rgb: endpoint {
            remote-endpoint = <&rgb_out_panel>;
          };
        };
      };
    };

    &backlight {
      pwms = <&pwm9 0 25000 0>;
      status = "okay";
    };
```

- Configure the `bus-format` property based on the display modes supported by the panel. This information is typically found in the panel datasheet under pin definitions and in the panel driver IC documentation regarding supported display modes. For macro definitions configured in the DTS, refer to the kernel file `include/uapi/linux/media-bus-format.h`. The mapping to hardware connections is as follows:

Display Mode	Bus Format	Cycles Per Pixel
RGB888 (24bit)	MEDIA_BUS_FMT_RGB888_1X24	1
RGB666 (18bit)	MEDIA_BUS_FMT_RGB666_1X18	1
RGB666_CPADHI (18bit)	MEDIA_BUS_FMT_RGB666_1X24_CPADHI	1
RGB565 (16bit)	MEDIA_BUS_FMT_RGB565_1X16	1
RGB565_CPADHI (16bit)	MEDIA_BUS_FMT_RGB565_1X24_CPADHI	1
RGB3x8 (8bit)	MEDIA_BUS_FMT_RGB888_3X8 MEDIA_BUS_FMT_RGB888_3X8	3
RGB4x8(8bit)	MEDIA_BUS_FMT_RGB888_DUMMY_4X8 MEDIA_BUS_FMT_BGR888_DUMMY_4X8	4

- The `pwms` configuration in the `backlight` node must be modified according to the actual hardware connections. Ensure the backlight is properly enabled before displaying images. For details, refer to the PWM module documentation: *Rockchip_Developer_Guide_Linux_PWM*.
- The `enable-gpios` / `reset-gpios` and `enable-delay-ms` / `reset-delay-ms` / `prepare-delay-ms` / `unprepare-delay-ms` / `disable-delay-ms` configurations must be adjusted based on the power-on/off and reset timing requirements specified in the panel datasheet, as well as the actual hardware circuit design.
- (Optional) The `enable` pin is typically used to enable power supply to the panel; its GPIO configuration depends on the specific design of the power circuit.
- (Optional) The `reset` pin is usually directly exposed on the panel side, with reset trigger conditions described in the datasheet; its GPIO configuration depends on the specific design of the reset circuit.
- (Optional) Configure `enable-delay-ms`, `reset-delay-ms`, `prepare-delay-ms`, `unprepare-delay-ms`, and `disable-delay-ms` according to the power/reset/signal timing requirements specified in the datasheet.
- The `display-timings` node: The panel datasheet provides recommended timing configurations. Users may also fine-tune these values within the specified upper and lower threshold ranges based on specific application requirements. The figure below shows an example panel node configuration alongside the corresponding panel datasheet:

ITEM		SYMBOL	MIN.	TYP.	MAX.	UNIT	Note
DE MODE	Dot Clock	1/tCLK	45	51.2	57	MHz	
	DCLK pulse duty	Tcwh	40	50	60	%	
	Horizontal total Time	tH	1324	1344	1364	tCLK	
	Horizontal effective Time	tHA		1024		tCLK	
	Horizontal Blank Time	tHB	300	320	340	tCLK	
	Vertical total Time	tV	625	635	645	tH	
	Vertical effective Time	tVA		600		tH	
	Vertical Blank Time	tVB	25	35	45	tH	
SYNC MODE	Horizontal total Time	TH	1324	1344	1364	tCLK	
	Horizontal Pulse Width	Thpw		20	-	tCLK	thb + thpw =160DCLK is fixed
	Horizontal Back Porch	Thb		140	-	tCLK	
	Horizontal Front Porch	Thfp	140	160	180	tCLK	
	Horizontal effective Time	THA		1024		tCLK	
	Vertical total Time	TV	625	635	645	tH	
	Vertical Pulse Width	Tvpw		3	-	th	tvpw + tvb =23th is fixed
	Vertical Back Porch	Tvb	-	20	-	th	
	Vertical Front Porch	Tvfp	2	12	22	th	
	Vertical Valid	Tvd		600		th	

The DRM framework's description of the `display_timing` structure and its variables can be found in the file `include/video/display_timing.h`, as shown below:

```
/*
 * Single "mode" entry. This describes one set of signal timings a display can
 * have in one setting. This struct can later be converted to struct videomode
 * (see include/video/videomode.h). As each timing_entry can be defined as a
 * range, one struct display_timing may become multiple struct videomodes.
 *
 * Example: hsync active high, vsync active low
 *
 *               Active Video
 * Video  _____________________XXXXXXXXXXXXXXXXXXXXXX_____________________
 *    |<- sync ->|<- back ->|<----- active ----->|<- front ->|<- sync.
 *    |      |   porch  |           |   porch   |
 *
 * HSync _|¯¯¯¯¯¯¯¯¯¯|_____________________________________|¯¯¯¯¯¯¯¯
 *
 * VSync ¯|_________|¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯¯|________
 */
struct display_timing {
    struct timing_entry pixelclock;

    struct timing_entry hactive;       /* hor. active video */
    struct timing_entry hfront_porch;  /* hor. front porch */
    struct timing_entry hback_porch;   /* hor. back porch */
    struct timing_entry hsync_len;     /* hor. sync len */

    struct timing_entry vactive;       /* ver. active video */
    struct timing_entry vfront_porch;  /* ver. front porch */
    struct timing_entry vback_porch;   /* ver. back porch */
    struct timing_entry vsync_len;     /* ver. sync len */

    enum display_flags flags;      /* display flags */
};
```

- For the RGB/MCU interface, the `clock-frequency` property value under the `display-timings` node determines the frame rate output by the VOP. The calculation requires the DRM framework's timing attributes `htotal` and `vtotal`. The following excerpt is provided for reference; see `include/drm/drm_modes.h` for details.

The frame rate calculation can refer to the implementation of the `drm_mode_vrefresh()` function in `drivers/gpu/drm/drm_modes.c`. Let the frame rate be fr; the calculation formula is:

$$fr = \frac{clock}{htotal \times vtotal}$$

```
/**
 * struct drm_display_mode - DRM kernel-internal display mode structure
 * @hdisplay: horizontal display size
 * @hsync_start: horizontal sync start
 * @hsync_end: horizontal sync end
 * @htotal: horizontal total size
 * @hskew: horizontal skew?!
 * @vdisplay: vertical display size
 * @vsync_start: vertical sync start
 * @vsync_end: vertical sync end
 * @vtotal: vertical total size
 * @vscan: vertical scan?!
.....
 *
 * The horizontal and vertical timings are defined per the following diagram.
 *
 * :
 *
 *
 *           Active             Front        Sync         Back
 *          Region              Porch                     Porch
 *    <----------------------><---------------><-------------><-------------->
 *      //////////////////////|
 *     ////////////////////// |
 *    //////////////////////  |.................              .................
 *                                 _______________
 *    <----- [hv]display ----->
 *    <------------ [hv]sync_start ------------>
 *    <---------------------- [hv]sync_end ---------------------->
 *    <------------------------------- [hv]total ------------------------------->*
 *
.....
 */

/**
 * drm_mode_vrefresh - get the vrefresh of a mode
 * @mode: mode
 *
 * Returns:
 * @modes's vrefresh rate in Hz, rounded to the nearest integer. Calculates the
 * value first if it is not yet set.
 */
```

```
int drm_mode_vrefresh(const struct drm_display_mode *mode)
{
    unsigned int num, den;

    if (mode->htotal == 0 || mode->vtotal == 0)
        return 0;

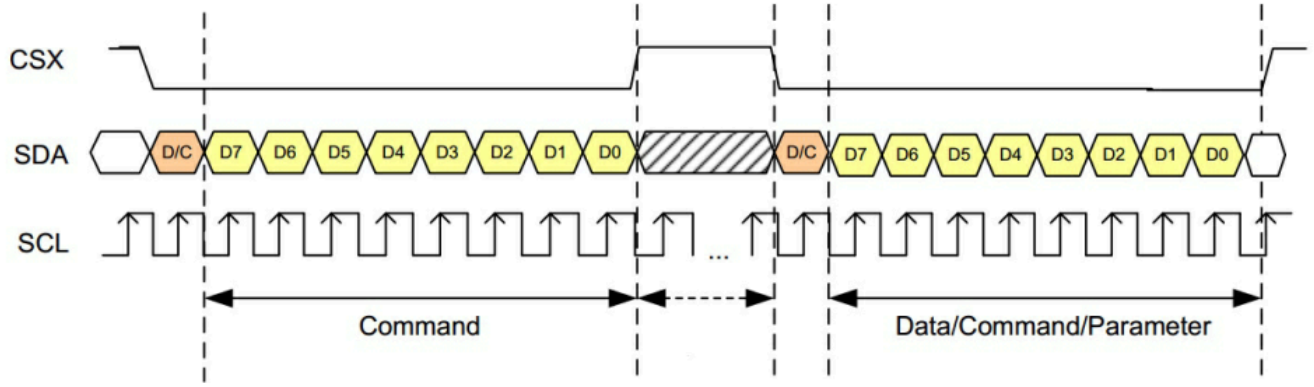
    num = mode->clock;
    den = mode->htotal * mode->vtotal;

    if (mode->flags & DRM_MODE_FLAG_INTERLACE)
        num *= 2;
    if (mode->flags & DRM_MODE_FLAG_DBLSCAN)
        den *= 2;
    if (mode->vscan > 1)
        den *= mode->vscan;

    return DIV_ROUND_CLOSEST_ULL(mul_u32_u32(num, 1000), den);
}
EXPORT_SYMBOL(drm_mode_vrefresh);
```

4.2.1 SPI Initialization Configuration

Some RGB panels require the main controller to send commands via the SPI interface to complete the initialization process. The file `drivers/gpu/drm/panel/panel-simple.c` supports the 3-wire 9-bit serial interface protocol:



Since the SPI module on the RK platform supports data transfer granularities of 4/8/16 bits, and the aforementioned protocol uses 9-bit data units, GPIO-simulated SPI is implemented in the panel-simple Driver.

Typical configuration for kernel version 5.10 and above:

```
spi_gpio: spi-gpio {
    compatible = "spi-gpio";
    #address-cells = <0x1>;
    #size-cells = <0x0>;
    pinctrl-names = "default";
    pinctrl-0 = <&spi_gpio_pins>;
    spi-delay-us = <10>;
```

```
status = "okay";

sck-gpios = <&gpio4 RK_PA5 GPIO_ACTIVE_HIGH>;
miso-gpios = <&gpio4 RK_PA7 GPIO_ACTIVE_HIGH>;
mosi-gpios = <&gpio4 RK_PA6 GPIO_ACTIVE_HIGH>;
cs-gpios = <&gpio4 RK_PA4 GPIO_ACTIVE_HIGH>;
num-chipselects = <1>;

/*
 * 320x480 RGB/MCU screen K350C4516T
 */
panel: panel {
    compatible = "simple-panel-spi";
    reg = <0>;
    bus-format = <MEDIA_BUS_FMT_RGB666_1X18>;
    backlight = <&backlight>;
    enable-gpios = <&gpio3 RK_PA6 GPIO_ACTIVE_LOW>;
    enable-delay-ms = <20>;
    reset-gpios = <&gpio3 RK_PB0 GPIO_ACTIVE_LOW>;
    reset-delay-ms = <10>;
    prepare-delay-ms = <20>;
    unprepare-delay-ms = <20>;
    disable-delay-ms = <20>;
    init-delay-ms = <10>;
    width-mm = <217>;
    height-mm = <136>;
    rockchip,cmd-type = "spi";
    status = "okay";

    // type:0 is cmd, 1 is data
    panel-init-sequence = [
        /* type delay num val1 val2 val3 */
        00  00  01  e0
        01  00  01  00
        ......
        00  78  01  11
        00  00  01  29
    ];

    panel-exit-sequence = [
        //type delay num val1 val2 val3
        00  0a  01  28
        00  78  01  10
    ];

    display-timings {
        native-mode = <&kd050fwfba002_timing>;

        kd050fwfba002_timing: timing0 {
            /*
             * 10453500  for RGB666(18bit)
             */
            clock-frequency = <10453500>;
```

```
            hactive = <320>;
            vactive = <480>;
            hback-porch = <10>;
            hfront-porch = <5>;
            vback-porch = <10>;
            vfront-porch = <5>;
            hsync-len = <10>;
            vsync-len = <10>;
            hsync-active = <0>;
            vsync-active = <0>;
            de-active = <0>;
            pixelclk-active = <1>;
        };
    };

    port {
        panel_in_rgb: endpoint {
            remote-endpoint = <&rgb_out_panel>;
        };
    };
  };
};
```

- Enable the configuration option CONFIG_SPI_GPIO and configure the corresponding sck-gpios, miso-gpios, mosi-gpios, and cs-gpios according to the hardware design. For details, refer to the Driver at drivers/spi/spi-gpio.c.
- The panel node configuration is basically the same as described above, with the following points to note:
    - Change compatible to "simple-panel-spi".
    - Set rockchip,cmd-type to "spi".
    - Fill in the corresponding init/deinit sequences in panel-init- sequence/panel-exit-sequence.
- For kernel versions 4.19 and below, the SDI/SCL/CS pins used by the SPI RGB display (**Note: SDI is actually not supported; the SDI configured in the DTS is actually the hardware SDO. This is an error in early versions**) are simulated using GPIOs directly in the simple-panel Driver, without relying on third-party SPI Drivers. Example:

```
panel: panel {
  compatible = "simple-panel";
  ......
  spi-sdi-gpios = <&gpio1 RK_PC7 GPIO_ACTIVE_HIGH>;
  spi-scl-gpios = <&gpio1 RK_PD0 GPIO_ACTIVE_HIGH>;
  spi-cs-gpios = <&gpio1 RK_PD1 GPIO_ACTIVE_HIGH>;
  ......
  rockchip,cmd-type = "spi";

  /* type:0 is cmd, 1 is data */
  panel-init-sequence = [
    /* type delay num val1 val2 val3 */
     00  00  01  e0
     ......
```

```
        00  78  01  11
        00  00  01  29
    ];

    panel-exit-sequence = [
        /* type delay num val1 val2 val3 */
        00  0a  01  28
        00  78  01  10
    ];
    ......
};
```

- Reference configurations for SPI RGB displays across different kernel versions are as follows:
    - Kernel 4.19 and below: arch/arm64/boot/dts/rockchip/rk3308-evb-ext-v10.dtsi
    - Kernel 5.10 and above: arch/arm64/boot/dts/rockchip/rk3562-evb1-lp4x-v10-rgb-k350c4516t.dts

4.3 RGB Interface

The RGB Driver corresponds to the file drivers/gpu/drm/rockchip/rockchip_rgb.c. Refer to the following DTS configuration:

```
&rgb {
    status = "okay";
    pinctrl-0 = <&rgb666_pins>;

    ports {
        port@1 {
            reg = <1>;

            rgb_out_panel: endpoint {
                remote-endpoint = <&panel_in_rgb>;
            };
        };
    };
};

//VOP 1.0
&rgb_in_vop {
    status = "okay";
};

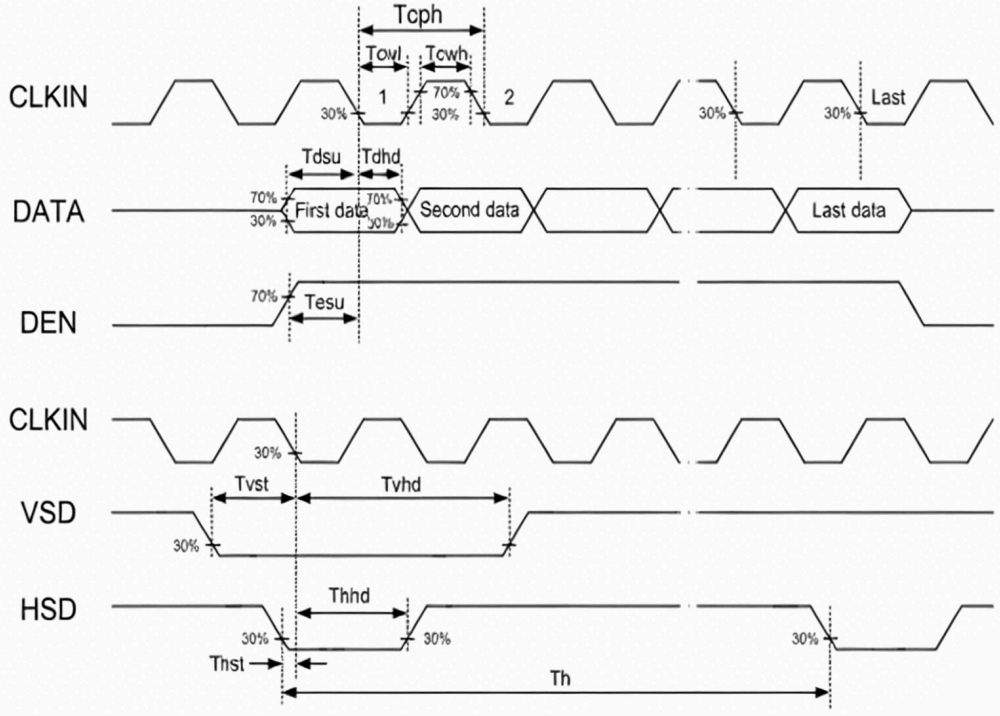
//VOP 2.0
&rgb_in_vp0 {
    status = "okay";
};
```

- Configuration for RGB interface nodes differs between VOP 1.0 and VOP 2.0 architectures. Refer to the following sample configurations:
    - VOP 1.0: `arch/arm/boot/dts/rv1106-evb-ext-rgb-v10.dtsi` .

- VOP 2.0: Refer to `arch/arm64/boot/dts/rockchip/rk3562-evb1-lp4x-v10-rgb-FX070-DHM11BOE-A.dts`.
- Pinctrl configuration must be determined based on actual hardware connections. Definitions for various pin mappings can be found in the `rkxxxx-pinctrl.dtsi` or `rvxxxx-pinctrl.dtsi` files.

4.3.1 RGB Delayline Configuration

Parameter	Symbol	Spec.			Unit	Condition
		Min.	Typ.	Max.		
DVDD power on slew rate	TPOR	-	-	20	ms	From 0V to 90% DVDD
RSTB pulse width	TRst	50	-	-	us	DCLK=65MHz
DCLK cycle time	Tcph	14	-	-	ns	
DCLK pulse duty	Tcwh	40	50	60	%	
VSD setup time	Tvst	5	-	-	ns	
VSD hold time	Tvhd	5	-	-	ns	
HSD setup time	Thst	5	-	-	ns	
HSD hold time	Thhd	5	-	-	ns	
Data setup time	Tdsu	5	-	-	ns	D0[7:0, D1[7:0], D2[7:0 to DCLK]]
Data hold time	Tdhd	5	-	-	ns	D0[7:0, D1[7:0], D2[7:0 to DCLK]]
DEN setup time	Tesu	5	-	-	ns	
DEN hold time	Tehd	5	-	-	ns	



The figure above is excerpted from the datasheet of panel FX070-DHM11BOE-A in the Panel Configuration section. When parameters such as Tdsu and Tdhd under default configurations fail to meet timing requirements, platforms supporting delayline adjustment (see RK Platform Support Status) can fine-tune the phase between CLKIN and DATA using the "rockchip,delayline-num" property under the RGB node:

```
&rgb {
    rockchip,delayline-num = <10>;
};
```

The unit of the delay line is approximately 0.05 ns (subject to external factors such as temperature; refer to measured values for accuracy). The above configuration delays the CLKIN output by 10 * 0.05 = 0.5 ns.

4.4 MCU Interface

For the MCU interface and MCU panel Driver, refer to `drivers/gpu/drm/rockchip/rockchip_rgb.c`. The DTS configuration is basically the same as that of the RGB interface, with the addition of flags and timing for switching to MCU mode. See the reference configuration below:

```
&rgb {
    status = "okay";
    rockchip,data-sync-bypass;
    pinctrl-names = "default";
    /*
     * rgb3x8_pins_m0/rgb3x8_pins_m1 for RGB3x8(8bit)
     * rgb565_pins for RGB565(16bit)
     */
    pinctrl-0 = <&rgb565_pins>;

    /*
     * 320x480 RGB/MCU screen K350C4516T
     */
    mcu_panel: mcu-panel {
        /*
         * MEDIA_BUS_FMT_RGB888_3X8  for RGB3x8(8bit)
         * MEDIA_BUS_FMT_RGB565_1X16 for RGB565(16bit)
         */
        bus-format = <MEDIA_BUS_FMT_RGB565_1X16>;
        backlight = <&backlight>;
        enable-gpios = <&gpio1 RK_PA3 GPIO_ACTIVE_LOW>;
        enable-delay-ms = <20>;
        reset-gpios = <&gpio1 RK_PA4 GPIO_ACTIVE_LOW>;
        reset-value = <0>;
        reset-delay-ms = <10>;
        prepare-delay-ms = <20>;
        unprepare-delay-ms = <20>;
        disable-delay-ms = <20>;
        init-delay-ms = <10>;
        width-mm = <217>;
        height-mm = <136>;

        // type:0 is cmd, 1 is data
        panel-init-sequence = [
            //type delay num val1 val2 val3
            ......
            01   00  01  55 /*
                          * interface pixel format:
                          * 66 for RGB3x8(8bit)
                          * 55 for RGB565(16bit)
                          */
            ......
```

```
        01  00  01  a0 /*
                     * frame rate control:
                     * 70 (45hz) for RGB3x8(8bit)
                     * a0 (60hz) for RGB565(16bit)
                     */
        ......
        01  00  01  02 /*
                     * display function control:
                     * 32 for RGB
                     * 02 for MCU
                     */
        00  78  01  11
        00  32  01  29
        00  00  01  2c
    ];

    panel-exit-sequence = [
      //type delay num val1 val2 val3
        00  0a  01  28
        00  78  01  10
    ];

    display-timings {
      native-mode = <&kd050fwfba002_timing>;

      kd050fwfba002_timing: timing0 {
        /*
         * 7840125  for frame rate 45Hz
         * 10453500 for frame rate 60Hz
         */
        clock-frequency = <10453500>;
        hactive = <320>;
        vactive = <480>;
        hback-porch = <10>;
        hfront-porch = <5>;
        vback-porch = <10>;
        vfront-porch = <5>;
        hsync-len = <10>;
        vsync-len = <10>;
        hsync-active = <0>;
        vsync-active = <0>;
        de-active = <0>;
        pixelclk-active = <1>;
      };
    };

    port {
        panel_in_rgb: endpoint {
          remote-endpoint = <&rgb_out_panel>;
        };
      };
    };
```

```
        ports {
            rgb_out: port@1 {
                reg = <1>;
                #address-cells = <1>;
                #size-cells = <0>;

                rgb_out_panel: endpoint@0 {
                    reg = <0>;
                    remote-endpoint = <&panel_in_rgb>;
                };
            };
        };
    };

//VOP 1.0
&rgb_in_vop {
    status = "okay";
};

&vop {
    status = "okay";

    /*
     * Default config is as follows:
     *
     * mcu-pix-total = <9>;
     * mcu-cs-pst = <1>;
     * mcu-cs-pend = <8>;
     * mcu-rw-pst = <2>;
     * mcu-rw-pend = <5>;
     * mcu-hold-mode = <0>; // default set to 0
     *
     * To increase the frame rate, reduce all parameters because
     * the max dclk rate of mcu is 150M in rv1103/rv1106.
     */
    mcu-timing {
        mcu-pix-total = <5>;
        mcu-cs-pst = <1>;
        mcu-cs-pend = <4>;
        mcu-rw-pst = <2>;
        mcu-rw-pend = <3>;

        mcu-hold-mode = <0>; // default set to 0
    };
};

//VOP 2.0
&rgb_in_vp0 {
    status = "okay";
};

&vp0 {
    status = "okay";
```

```
  /*
   * Default config is as follows:
   *
   * mcu-pix-total = <9>;
   * mcu-cs-pst = <1>;
   * mcu-cs-pend = <8>;
   * mcu-rw-pst = <2>;
   * mcu-rw-pend = <5>;
   * mcu-hold-mode = <0>; // default set to 0
   *
   * To increase the frame rate, reduce all parameters because
   * the max dclk rate of mcu is 150M in rk3562.
   */
  mcu-timing {
    mcu-pix-total = <5>;
    mcu-cs-pst = <1>;
    mcu-cs-pend = <4>;
    mcu-rw-pst = <2>;
    mcu-rw-pend = <3>;

    mcu-hold-mode = <0>; // default set to 0
  };
};
```

- For VOP 1.0 and VOP 2.0 architectures, the configuration of MCU interface-related nodes differs. Refer to the following sample configurations:
  - VOP 1.0: arch/arm/boot/dts/rv1106-evb-ext-mcu-v10.dtsi.
  - VOP 2.0: Refer to arch/arm64/boot/dts/rockchip/rk3562-evb1-lp4x-v10-mcu-k350c4516t.dts.
- The Driver switches between MCU and RGB interface modes based on the rockchip,data-sync-bypass property under the rgb node. If this property is absent, the default mode is RGB interface; enabling it switches to MCU interface.
- In kernel-5.10 and later versions, it is recommended to place the panel configuration for the MCU interface under the rgb node. For kernel-4.19 and earlier versions, initialize the panel as an independent node using the simple-panel Driver. When migrating a panel node originally intended for the simple-panel Driver under the rgb node, note the following:
  - Ensure the panel node is named mcu-panel, as the Driver identifies and parses MCU panel parameters based on this name.
  - The compatible property can be omitted and does not require configuration.
- MCU panels typically require an initialization sequence to configure display mode, pixel format, frame rate, etc. (specifics depend on the panel's Driver IC), as well as an exit sequence to ensure the panel powers off or enters low-power mode. Notes regarding the panel-init-sequence and panel-exit-sequence properties are as follows:
  - Sequences are provided by the panel manufacturer and usually need to be converted from C files into DTS configuration.
  - Each line of the sequence, from left to right, consists of: command/data type, delay time (ms), data length (bytes), and data.

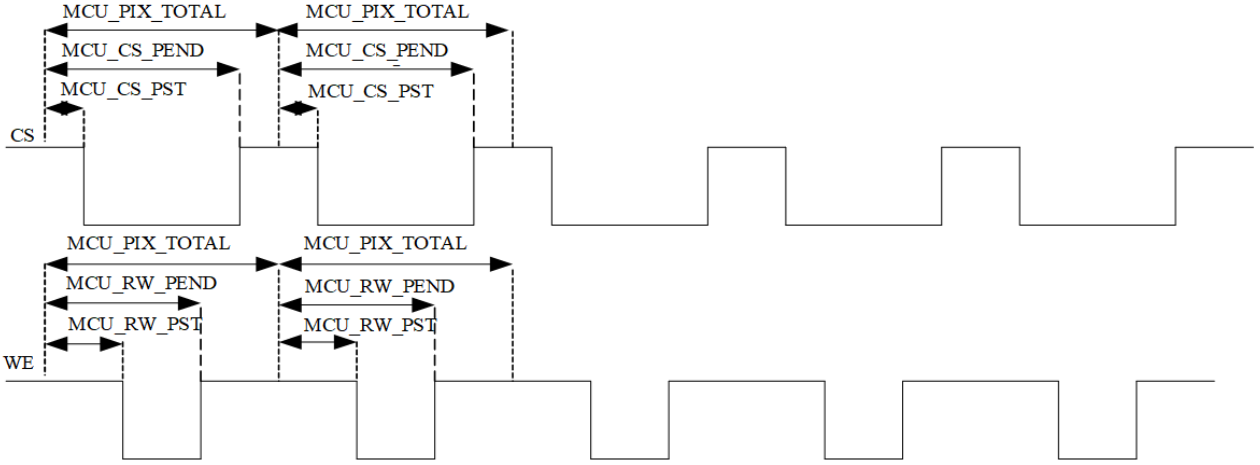
- **Frame rate configuration is typically performed during the initialization sequence phase** and must correspond to the frame rate calculated in Panel Configuration. Below is an excerpt from a sample Driver IC datasheet:

5.3.2. Frame Rate Control (In Normal Mode/Full Colors) (B1h)

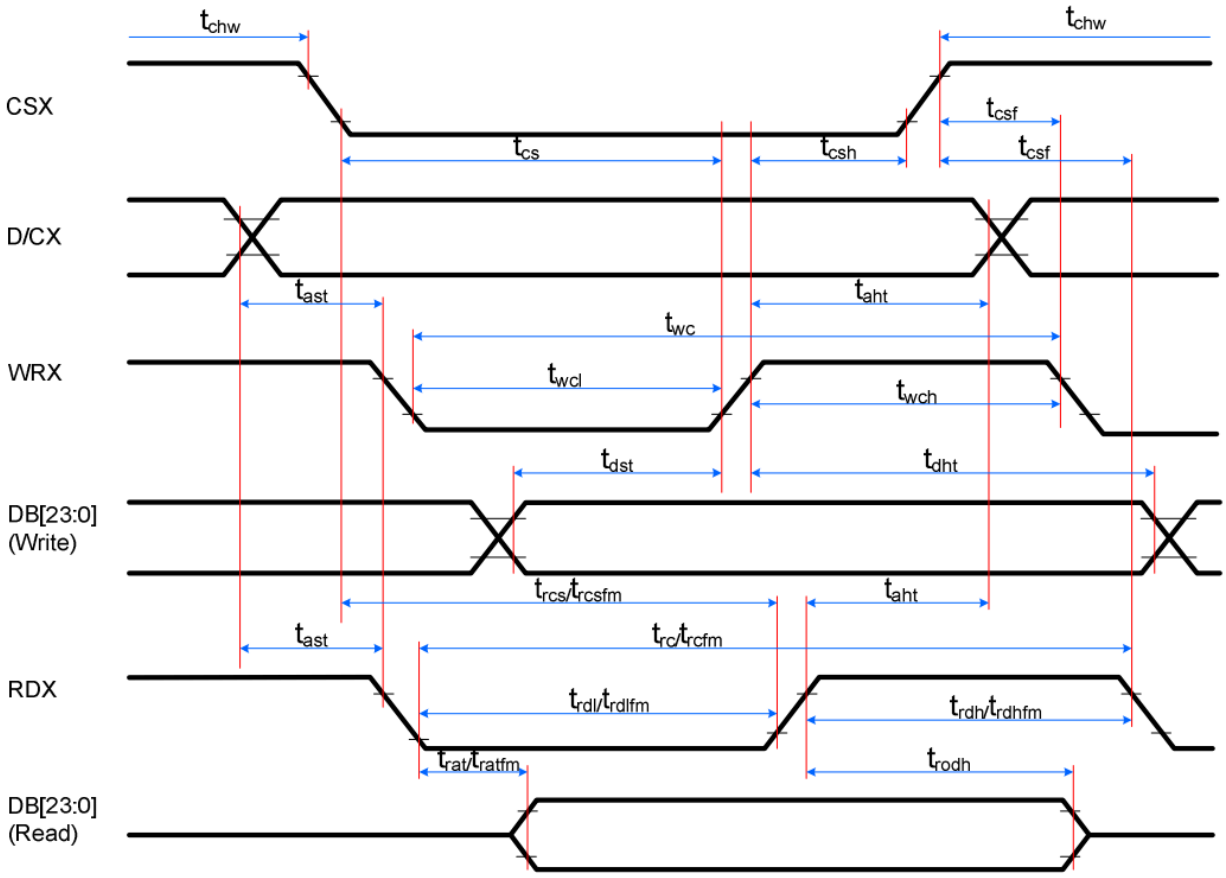
B1h	FRMCTR1 (Frame Rate Control (In Normal Mode/Full colors))												
	D/CX	RDX	WRX	D [23:8]	D7	D6	D5	D4	D3	D2	D1	D0	HEX
Command	0	1	↑	XX	1	0	1	1	0	0	0	1	B1h
1st Parameter	1	1	↑	XX	FRS [3:0]				0	0	DIVA [1:0]		B0h
2nd Parameter	1	1	↑	XX	0	0	0	RTNA [4:0]					11h

FRS [3:0]				CNT	Frame rate(Hz) Tearing Effect Line OFF(R34h)	Frame rate(Hz) Tearing Effect Line ON(R35h) VBP+VFP <24
0	0	0	0	37	28.78	27.64
0	0	0	1	35	30.38	29.17
0	0	1	0	33	32.17	30.89
0	0	1	1	31	34.18	32.82
0	1	0	0	29	36.46	35.01
0	1	0	1	27	39.06	37.51
0	1	1	0	25	42.07	40.40
0	1	1	1	23	45.57	43.76
1	0	0	0	21	49.71	47.74
1	0	0	1	19	54.69	52.52
1	0	1	0	17	60.76	58.35
1	0	1	1	15	68.36	65.65
1	1	0	0	13	78.13	75.03
1	1	0	1	11	91.15	87.53

- The `mcu-timing` property configures the timing of the CSN/WEN/REN control signals for the MCU interface in normal mode. Each attribute corresponds to the time intervals shown in the figures below. The detailed timing diagrams and specific requirements for each interval are extracted from the panel module datasheet referenced in the example DTS configuration. The following sections describe how to configure the attributes in `mcu-timing` and the `clock-frequency` attribute in `display-timings`.







Signal	Symbol	Parameter	min	max	Unit	Description
DCX	tast	Address setup time	0	-	ns	-
	that	Address hold time (Write/Read)	0	-	ns	-
CSX	tchw	CSX "H" pulse width	0	-	ns	-
	tcs	Chip Select setup time (Write)	15	-	ns	-
	trcs	Chip Select setup time (Read ID)	45	-	ns	-
	trcsfm	Chip Select setup time (Read FM)	355	-	ns	-
	tcsf	Chip Select Wait time (Write/Read)	0	-	ns	-
WRX	twc	Write cycle	30	-	ns	-
	twrh	Write Control pulse H duration	15	-	ns	-
	twrl	Write Control pulse L duration	15	-	ns	-
RDX (FM)	trcfm	Read Cycle (FM)	450	-	ns	When read from Frame Memory
	trdhfm	Read Control H duration (FM)	90	-	ns	
	trdlfm	Read Control L duration (FM)	355	-	ns	
RDX (ID)	trc	Read cycle (ID)	160	-	ns	When read ID data
	trdh	Read Control pulse H duration	90	-	ns	
	trdl	Read Control pulse L duration	45	-	ns	
DB [23:0], DB [17:0], DB [15:0], DB [8:0], DB [7:0]	tdst	Write data setup time	10	-	ns	For maximum, CL=30pF For minimum, CL=8pF
	tdht	Write data hold time	10	-	ns	
	trat	Read access time	-	40	ns	
	tratfm	Read access time	-	340	ns	
	trod	Read output disable time	20	80	ns	

- Let the variable `ptotal` represent the value of `mcu-pix-total` in `mcu-timing`. According to the table, the panel datasheet requires that the duration of the MCU_PIX_TOTAL interval must be no less than t_{min}:

  (In the panel datasheet, t_{wc} = 40, while in the Driver IC datasheet, t_{wc} = 30; the larger of the two values is selected.)

$$t_{min} = T_{CHW} + T_{AST} + T_{WC} + T_{CHW} = 40(ns)$$

For the example panel, which supports both MEDIA_BUS_FMT_RGB888_3X8 and MEDIA_BUS_FMT_RGB565_1X16 modes, let the Cycles Per Pixel value (referring to splitting the data of one pixel into n cycles for transmission; see the Panel Configuration section for details) be denoted as cpp. The theoretical maximum frame rates for the two modes (denoted as $fr_{s\text{-}max}$ and $fr_{p\text{-}max}$, respectively) can be calculated as follows:

$$fr < \frac{1 * 1000000000}{htotal \times ptotal \times cpp \times t_{min}}$$

$$htotal = hactive + hback - porch + hfront - porch + hsync - len = 345$$

$$vtotal = vactive + vback - porch + vfront - porch + vsync - len = 505$$

$$fr_{s-max} < \frac{1 * 1000000000}{345 \times 505 \times 3 \times 40} \approx 47.83(Hz)$$

$$fr_{p-max} < \frac{1 * 1000000000}{345 \times 505 \times 1 \times 40} \approx 143.49(Hz)$$

Therefore, for the serial RGB3x8 mode, the frame rate fr_s = 45(Hz), and for the parallel RGB1x16 mode, the frame rate fr_p = 60(Hz).

(In the following variables, the subscript 's' denotes serial, and 'p' denotes parallel; this convention will not be repeated.)

The values of the clock-frequency property in the display-timings configuration for both modes can also be calculated as follows:

$$clock_s = fr_s \times htotal \times vtotal = 45 \times 345 \times 505 = 7840125$$

$$clock_p = fr_p \times htotal \times vtotal = 60 \times 345 \times 505 = 10453500$$

- Each platform's MCU interface has a maximum limit for dclk. For details, refer to the `mcu_max_dclk_rate` attribute in the Driver at `drivers/gpu/drm/rockchip/rockchip_rgb.c`, which is typically 150000000 Hz. The corresponding validation logic in the Driver can be found in the `rockchip_rgb_encoder_mode_valid()` function.

  Based on $dclk_{max}$ and the frame rate configured in the initialization sequence (denoted as variable fr, typically 60Hz), the maximum value of ptotal can be calculated:

$$ptotal_{max} < \frac{dclk_{max}}{cpp \times clock} - 1$$

$$ptotal_{s-max} < \frac{150000000}{3 \times 7840125} - 1 \approx 5.38$$

$$ptotal_{p-max} < \frac{150000000}{1 \times 10453500} - 1 \approx 13.35$$

  The actual ptotal takes the smaller of the two values, ptotal = 5. Other attribute values for mcu-timing can then be determined from the timing diagram.

- **Finally, the calculated actual values must be verified against the timing requirements specified in the datasheet.**

The actual dclk of the MCU interface is not equal to the clock-frequency property value in the display-timings configuration; it also depends on the mcu-pix-total configuration in mcu-timing and the Cycles Per Pixel value:

$$dclk = clock \times cpp \times (ptotal + 1)$$

$$dclk_s = 7840141122250$$

$$dclk_p = 62721000$$

The above dclk values do not exceed $dclk_{max}$. Additionally, the actual time duration of the MCU_PIX_TOTAL interval can be calculated:

$$t_s = \frac{(ptotal + 1) \times 1000000000}{dclk_s} \approx 42.52(ns)$$

$$t_p = \frac{(ptotal + 1) \times 1000000000}{dclk_p} \approx 95.66(ns)$$

The time interval t for the aforementioned MCU_PIX_TOTAL range satisfies the requirement of being greater than 40 ns.

4.4.1 MCU Bypass Timing Configuration

For early platforms supporting the MCU interface, such as RK3308 and PX30, the timing configuration for bypass mode is consistent with that of normal mode. Starting from the RK3562 platform and later, the IC design has been optimized. The Driver includes a set of default bypass mode timing parameters to meet the write/read operation requirements in most bypass mode scenarios. For details, refer to the relevant definitions of the `vop_mcu_bypass_cfg` structure in `drivers/gpu/drm/rockchip/rockchip_vop_reg.c` and `drivers/gpu/drm/rockchip/rockchip_vop2_reg.c`, as shown below:

```
static struct rockchip_mcu_timing rk3506_mcu_bypass_timing = {
    .mcu_pix_total = 26,
    .mcu_cs_pst = 3,
    .mcu_cs_pend = 24,
    .mcu_rw_pst = 6,
    .mcu_rw_pend = 15,
};

static const struct vop_mcu_bypass_cfg rk3506_mcu_bypass_cfg = {
    .timing = &rk3506_mcu_bypass_timing,
    .dclk_rate = 120000000,
};
```

If the default bypass mode timing does not meet the requirements of the application scenario, it can be specified in the device tree source (DTS) file (refer to arch/arm/boot/dts/rk3506g-evb1-v10-mcu-k350c4516t.dts):

```
&vop {
    mcu-timing {
```

```
        mcu-pix-total = <5>;
        mcu-cs-pst = <1>;
        mcu-cs-pend = <4>;
        mcu-rw-pst = <2>;
        mcu-rw-pend = <3>;

        mcu-hold-mode = <0>;
    };

    mcu-bypass-timing {
        mcu-pix-total = <9>;
        mcu-cs-pst = <1>;
        mcu-cs-pend = <8>;
        mcu-rw-pst = <2>;
        mcu-rw-pend = <7>;

        mcu-hold-mode = <0>;
    };
};
```

- Note that bypass mode may involve MCU read operations. Typically, the minimum timing constraint for MCU read exceeds that for MCU write (refer to the example datasheet in the previous chapter). Therefore, `mcu-bypass-timing` is usually calculated and configured based on the read timing constraints specified in the datasheet to simultaneously satisfy the timing requirements for both write and read operations.

4.4.2 MCU Frame Write/Read

In MCU normal mode, the frame write/read function enables command interaction between the host controller and the panel during the vertical blanking interval. This is commonly used to read panel status Registers, modify the display start position, perform static voltage testing, or meet specific panel requirements for updating the display start position on a per-frame basis.

To enable this function, configure the `panel-frame-sequence` property in the DTS. Refer to `arch/arm/boot/dts/rk3506g-evb1-v10-mcu-k350c4516t.dts` for an example:

```
&rgb {
    ......
    mcu_panel: mcu-panel {
        ......
        panel-frame-sequence = [
            //type delay num val1 val2 val3
              00  00  01  0a
              02  00  01  0a
              02  00  01  0a
              00  00  01  2c
        ];
        ......
    };
    ......
```

```
};
```

- Frame-to-frame write/read operations are implemented in `drivers/gpu/drm/rockchip/rockchip_rgb.c`:

```
static void mcu_interframe_work_func(struct work_struct *work)
{
    struct rockchip_mcu_panel *mcu_panel =
        container_of(work, struct rockchip_mcu_panel, interframe_work);
    struct drm_panel *panel = &mcu_panel->base;
    struct drm_display_mode *mode = mcu_panel->desc->mode;
    struct drm_crtc *crtc = mcu_panel->encoder->crtc;
    u32 timeout = DIV_ROUND_CLOSEST_ULL(1000, drm_mode_vrefresh(mode));
    int ret;

    ret = rockchip_drm_wait_vact_end(crtc, timeout);
    if (ret) {
        DRM_DEV_ERROR(panel->dev, "wait vact end timed out\n");
        return;
    }

    ret = rockchip_mcu_panel_xfer_mcu_cmd_seq(mcu_panel, mcu_panel->desc->frame_seq);
    if (ret) {
        DRM_DEV_ERROR(panel->dev, "failed to send frame cmds seq\n");
        return;
    }

    schedule_work(&mcu_panel->interframe_work);
}
```

If there are specific application requirements, the implementation of the aforementioned work can be modified accordingly.

4.4.3 MCU PSR Mode

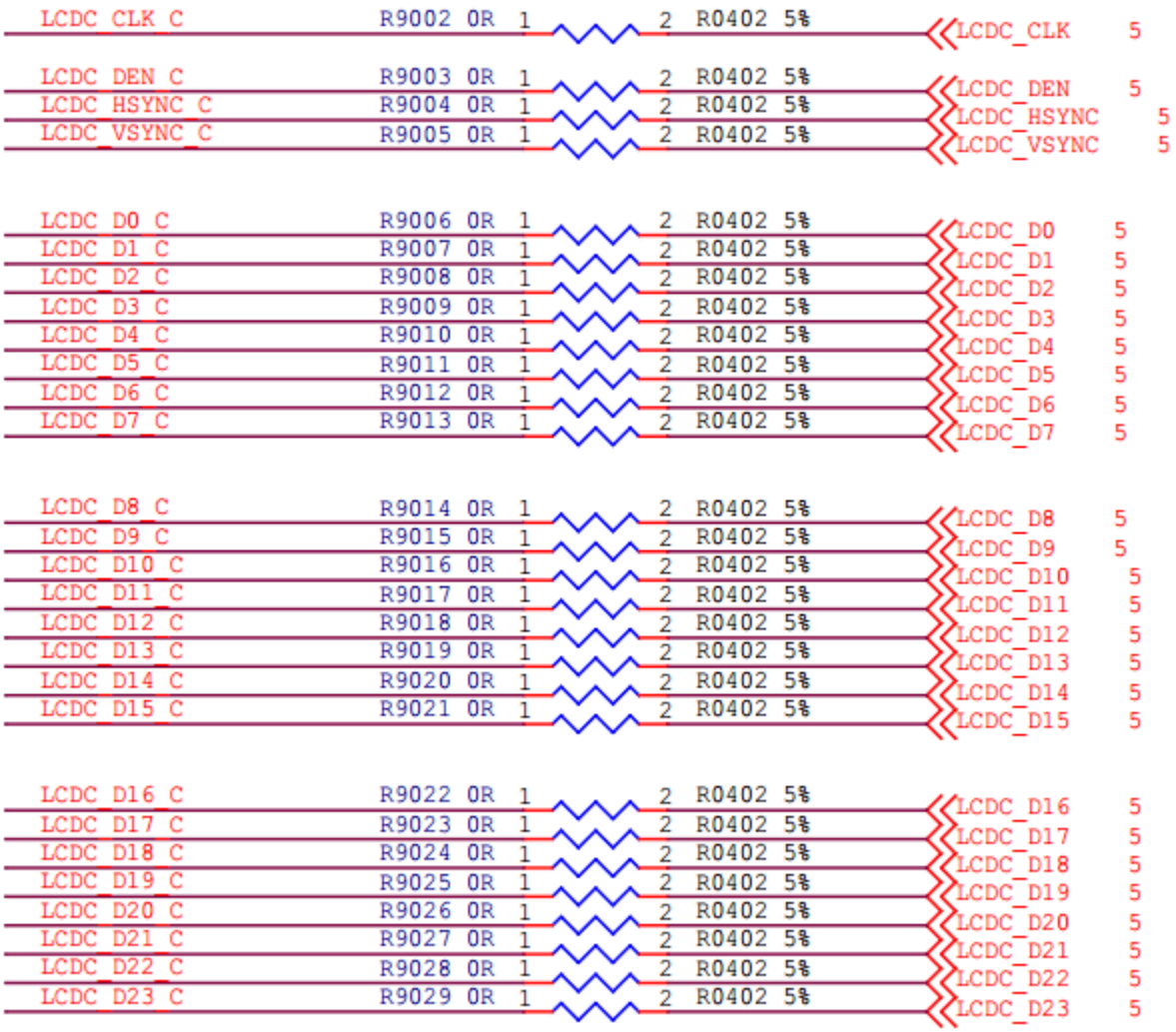
Typically, an MCU Panel contains internal RAM for storing display data. When no valid data is received from the TX side, the panel refreshes the same frame image in the RAM at a fixed frame rate (specified in the panel-init-sequence of the mcu_panel node). Leveraging this characteristic of MCU Panels, the RGB Driver implements the PSR (Panel Self Refresh) function in DBI Command Mode. In static display scenarios, both the VOP and RGB modules on the VOP -> MCU -> Panel display chain automatically enter a Low Power State, effectively reducing system power consumption. Under the Kernel-6.1 DTS configuration arch/arm64/boot/dts/rockchip/rk3576-test1-v10-mcu-k350c4516t.dts for an MCU -> Panel (K350C4516T) single-display static desktop scenario, enabling the PSR function has been measured to reduce average system power consumption by approximately 20 mW.

Note that this function is not supported by all MCU panels. It is necessary to confirm with the panel manufacturer the panel's behavior when no valid data is present on the TX side. Once confirmation is obtained that RAM data Self-refresh is supported, enable the PSR function via the "support-psr" property:

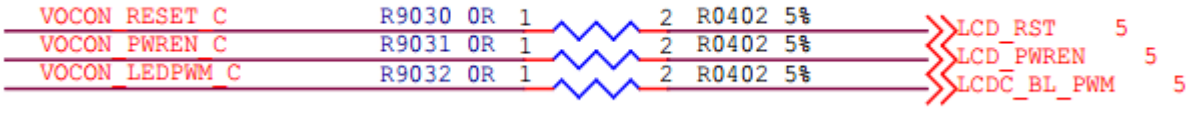
```
&rgb {
  support-psr;
};
```

5. Debugging Procedure

1. Verify the hardware connections for each pin of the RGB/MCU interface. Note that pin mapping may vary across different platforms; refer to the Hardware Connection section above for details.



2. Correctly configure the polarity and power-on timing of the enable/reset control pins, as well as the PWM channel used for the backlight, according to the panel datasheet. For MCU panels, also configure the correct initialization sequence.



3. Ensure the backlight is on, then verify whether the enable/reset control pins are at the correct logic

levels. If actual measurements still deviate from expected values, confirm that the iomux is correctly configured as GPIO.

6. FAQ

6.1 The RGB display can show images, but there is noise on the screen or display misalignment occurs

Answer:

1. Try inverting the dclk clock polarity by adjusting the `pixelclk-active` property under `display-timings`, which can shift the phase between dclk and data by 180°.

Taking `arch/arm64/boot/dts/rockchip/rk3562-evb1-lp4x-v10-rgb-FX070-DHM11BOE-A.dts` as an example:

```
display-timings {
    native-mode = <&fx070_dhm11boe_timing>;

    fx070_dhm11boe_timing: timing0 {
        clock-frequency = <50000000>;
        hactive = <1024>;
        vactive = <600>;
        hback-porch = <140>;
        hfront-porch = <160>;
        vback-porch = <20>;
        vfront-porch = <20>;
        hsync-len = <20>;
        vsync-len = <2>;
        hsync-active = <0>;
        vsync-active = <0>;
        de-active = <0>;
        pixelclk-active = <0>; // 1: flip, 0: no flip
    };
};
```

2. Attempt to adjust the delayline configuration to fine-tune the phase difference between dclk and data; refer to the RGB Delayline Configuration section above.